Implementation of Single Layer Perceptron

The following Python Code can be used to implement Single Layer Perceptron:

```
import  numpy as np
# Given Data
x  = np.array([1.0, 2.0])        # inputs
y = 1                            # target
w = np.array([0.2, -0.5])        # Initial weight
b = 0.1                          # Initial Bias
eta  = 0.1                       # Learning Rate
```

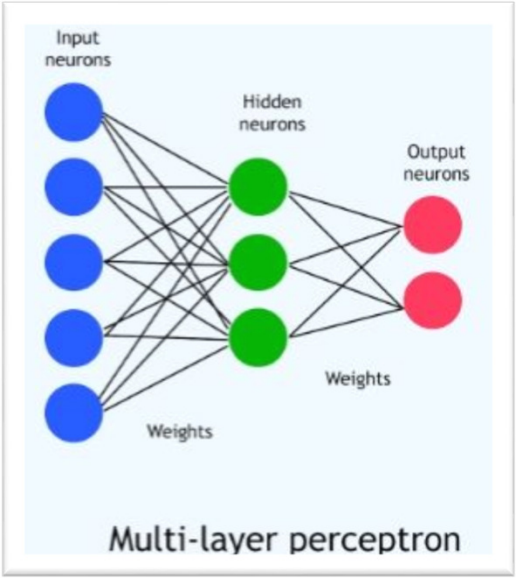
```
# Run for n epochs

for epoch in range(1,5):
    print("n Epoch  :" , epoch)
    net_input = np.dot(w,x) + b
    y_pred = 1 if net_input > 0 else 0
    error = y – y_pred

    print(f"Net Input:{net_input:.2f}")
    print(f"Predicated Output:{y_pred}")
    print(f" Error : {error}")

    w = w + eta * error * x
    b = b + eta * error
    print(f "Updated Weight : {w}")
    print(f "Updated Bias : {b:.2f}")
```

Types of Perceptron: Multi-Layer



Multi Layer Perceptron Model:

It is a supervised Feed forward neural network.

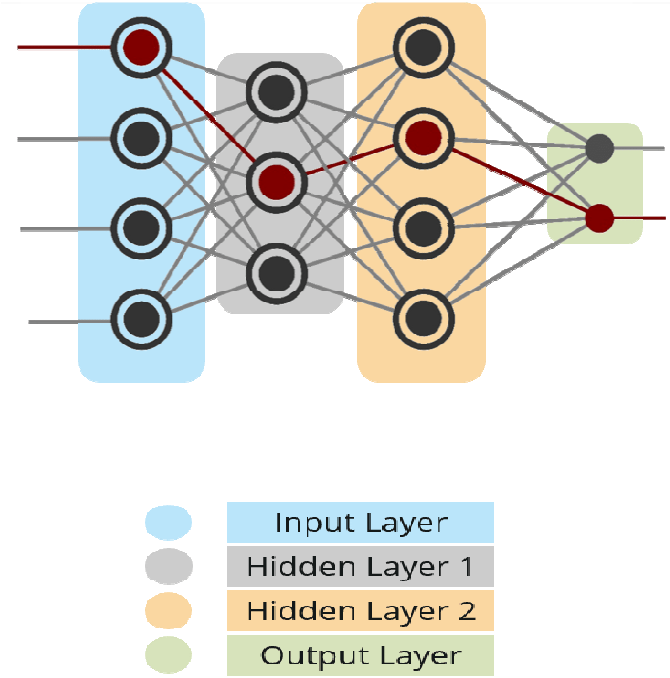
It is composed of multiple layers including an

Input Layer

Hidden Layer

Output Layer

where each layer contain a set of perceptron elements known as neurons.



Multi-Layer Perceptron: Use Cases

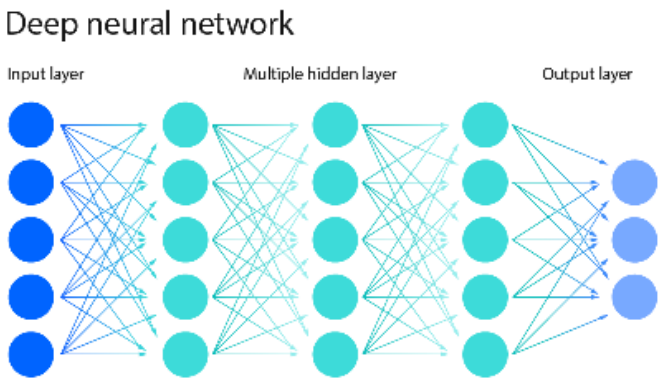
Multi Layer Perceptron Model is used to handle complex datasets that are not linearly separable.

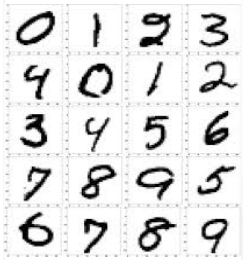
USE CASES:

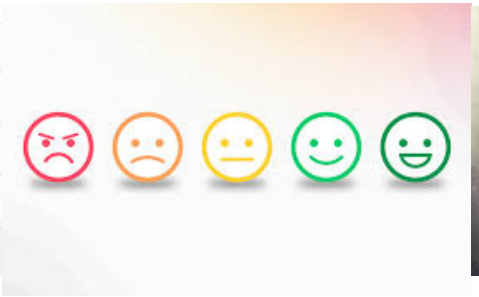
Image Recognition (Handwritten digit recognition)

Natural Language Processing (Sentiment Analysis)

Speech Recognition (Voice to text transcription)



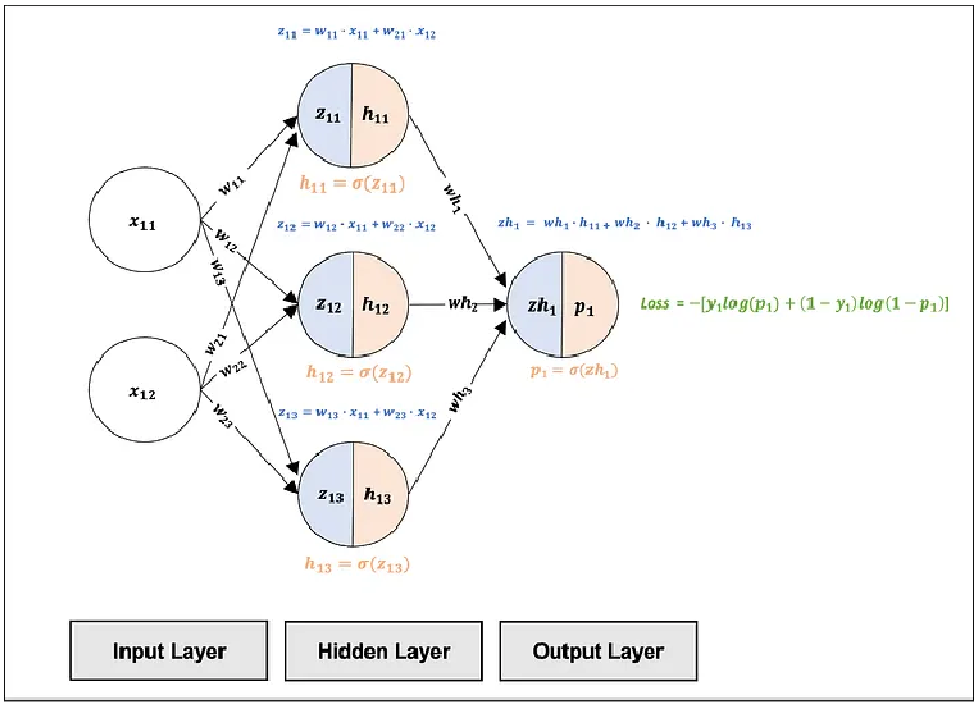




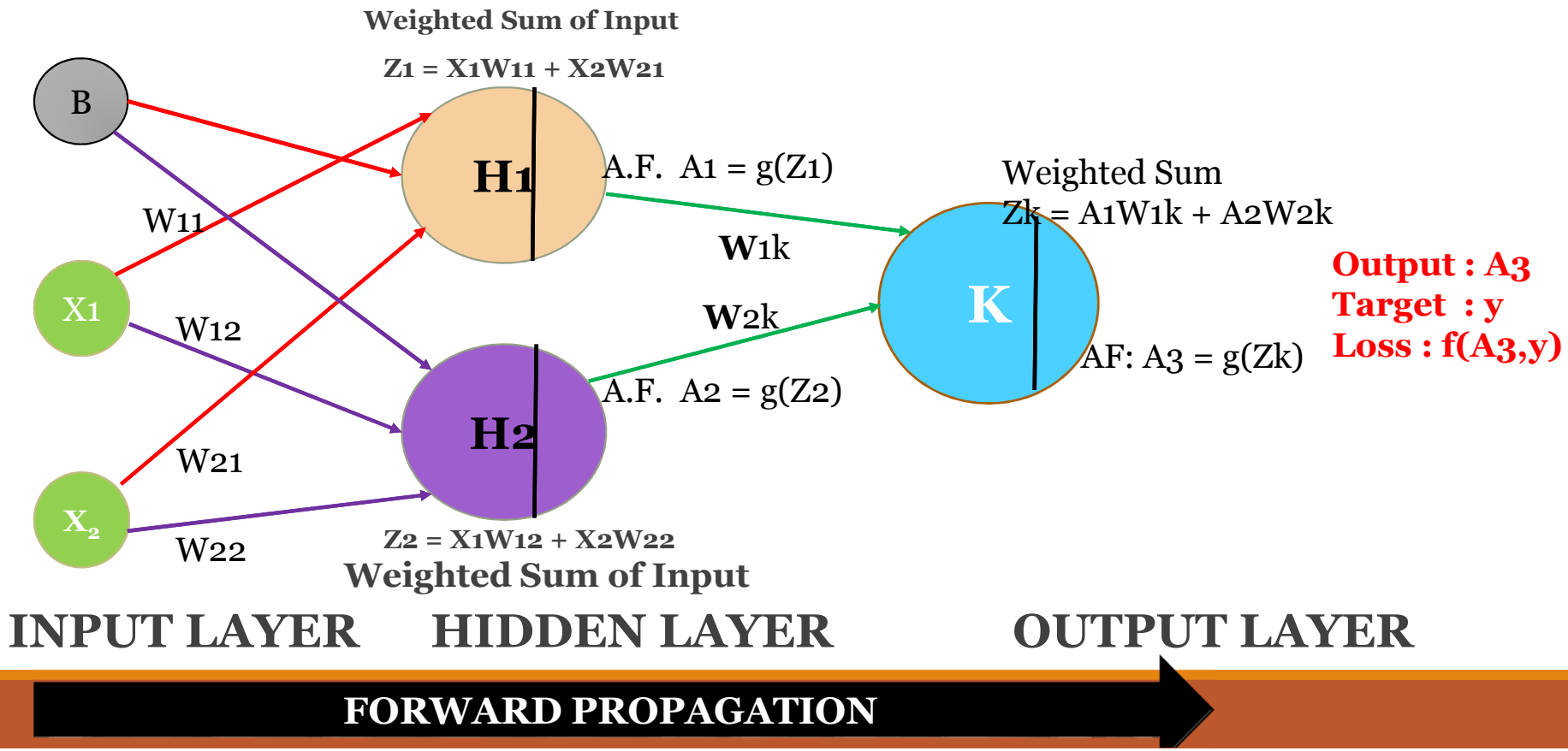
Working of Multi-Layer Perceptron

For the Perceptron Algorithms Necessary conditions:

- Input Features Must be Numeric Only.
- Non-Numeric features are required to be converted to numeric values before it's used in the Perceptron.
- Yes, image, audio, and video signals must be converted into a numeric format before being processed by a perceptron or any artificial neural network.



Working of Multi-Layer Perceptron



Multi-Layer Perceptron Numerical Problem

Suppose inputs $x_1=1$

$x_2=2$

Bias $b_1=0.5$

Activation function used: Sigmoid

$$g(x)=\frac{1}{1+e^{-x}}$$

Weights:

$w_{11} = 0.8$, $w_{21} = -0.5$

$w_{12} = 0.3$, $w_{22} = 1.0$

$w_{1k} = 1.2$, $w_{2k} = -0.7$

Neuron 1

$$z_1=w_{11}x_1+w_{21}x_2+b_1$$
$$= 0.8 * 1.0 + (-0.5) * 2.0 + 0.5$$
$$= 0.8 - 1.0 + 0.5 = 0.3$$
$$a_1=g(0.3)=\frac{1}{1+e^{-0.3}}=0.574442516812$$

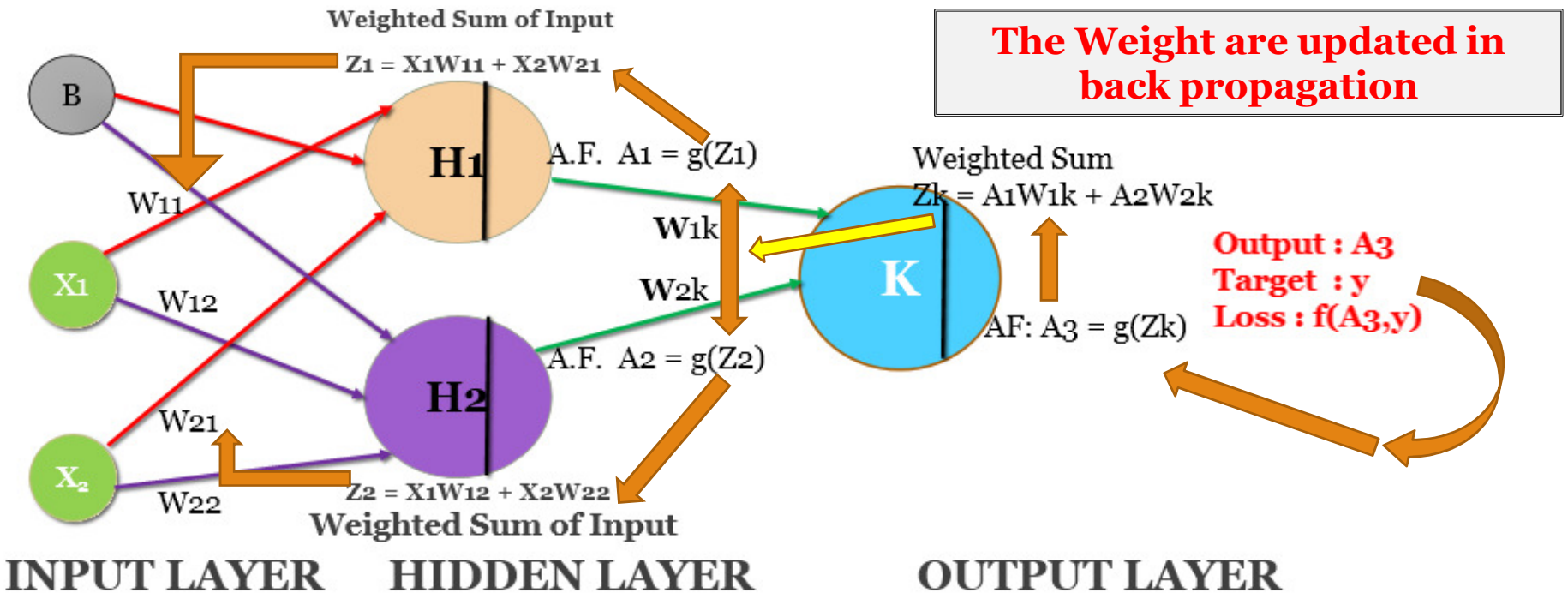
Neuron 2

$$z_2=w_{12}x_1+w_{22}x_2+b_1$$
$$= 0.3 * 1.0 + 1.0 * 2.0 + 0.5$$
$$= 0.3 + 2.0 + 0.5 = 2.8$$
$$a_2=g(2.8)=\frac{1}{1+e^{-2.8}}=0.942675824101$$

Output Neuron

$$z_3=w_{1k}a_1+w_{2k}a_2$$
$$=1.2*0.574442516812+(-0.7)*0.942675824101=0.029457943303$$
$$a_3=g(0.02945)=\frac{1}{1+e^{-0.02954}}=0.942675824101$$

Working of Multi-Layer Perceptron



The Weight are updated in back propagation

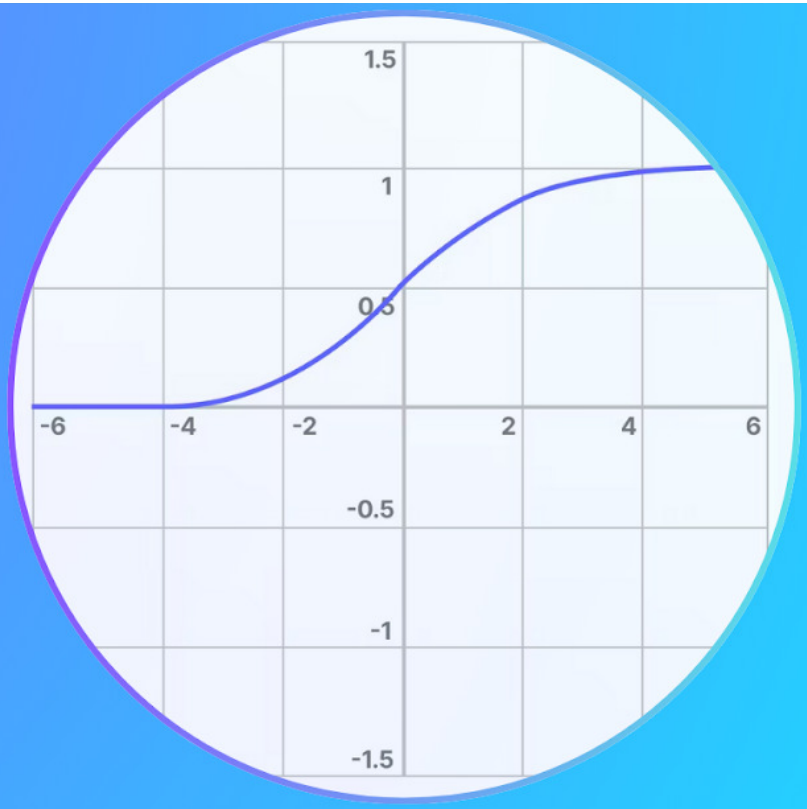
BACKWARD PROPAGATION

Applications of Multilayer perceptron

Applications of Multilayer perceptron:

1. **Classification** such as sentiment analysis, fraud detection
2. **Regression** such as score estimation
3. **NLP tasks** such as machine translation
4. **Anomaly Detection**
5. **Speech Recognition in virtual assistant systems** such as Siri, Alexa
6. **Computer Vision for** object identification, image segmentation
7. **Data analytics and data visualization**

Activation Functions in Neural Networks



ACTIVATION & LOSS FUNCTION

DEEP NEURAL NETWORK

Activation Function

Activation functions are an integral building block of **neural networks** that enable them to learn complex patterns in data.

1. In feed-forward propagation, the activation function is applied to a neurons output and it decides whether that output should be passed to the next layer or not.
2. It transforms the summed weighted input from the node into an output value to be fed to the next hidden layer or not.
3. They are the backbone of neural networks enabling them to capture non-linear relationships in data.

Without non-linearity, a neural network would only function as a simple linear regression model.

Types of Activation Function

Basically there are two types of activation functions:

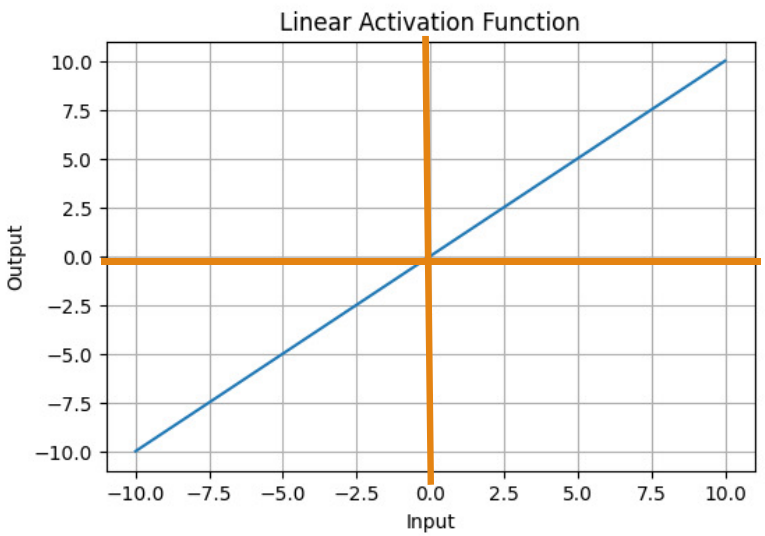
- **Linear Activation Function**
- **Non-Linear Activation Function:**
  - **Sigmoid**
  - **Tanh**
  - **Softmax**
  - **ReLU**
  - **Leaky ReLU**

Linear Activation Function

Linear Activation Function:

A linear activation function, or identity function, passes the weighted sum of inputs directly to the output without transformation ((f(x)=x)). It is primarily used in the output layer for regression problems to predict continuous, unbounded values ((-∞, +∞)). It is not used in hidden layers because it limits learning complex, non-linear patterns.

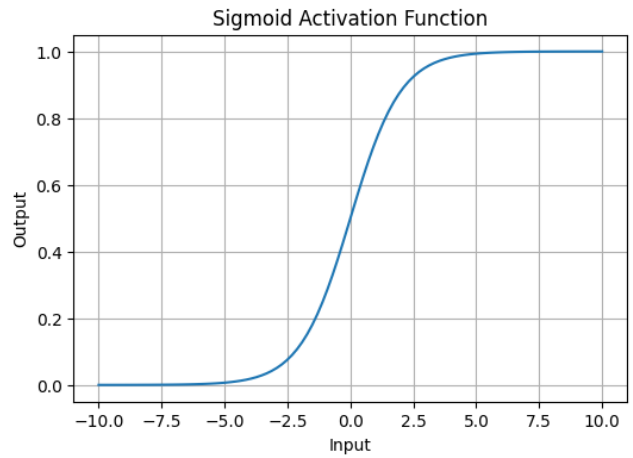
x	-10	-5	-1	0	1	5	10
f(x)	-10	-5	-1	0	1	5	-10



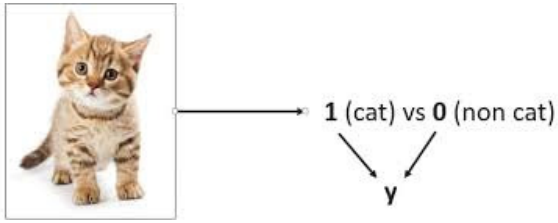
Sigmoid Function

Non-Linear Activation Function: Sigmoid

- It is characterized by 'S' shape.
- It is mathematically defined as $$f(x) = \frac{1}{1+e^{-x}}$$
- This formula ensures a smooth and continuous output that is essential for gradient-based optimization.
- It allows neural networks to handle and model complex patterns that linear equations cannot.
- The output ranges between 0 and 1, hence useful for binary classification.



x	-5	-3	-1	0	1	3	5
f(x)	0.0067	0.0047	0.268	0.5	0.731	0.956	0.993



Tanh Function

Tangent hyperbolic (Tanh) activation function:

The Non-Linear tanh (hyperbolic tangent) activation function is defined as:

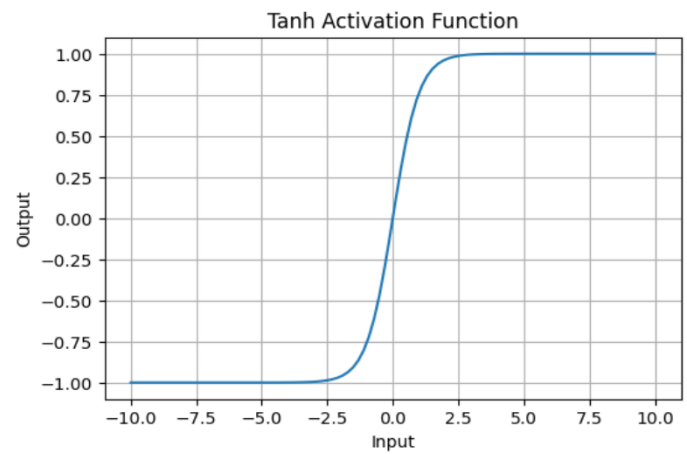
$$f(x) = \frac{e^x - -e^{-x}}{e^x + e^{-x}}$$

Value Range: Outputs values from -1 to +1.

Non-linear: Enables modeling of complex data patterns.

Use in Hidden Layers: Commonly used in hidden layers due to its zero-centered output, facilitating easier learning for subsequent layers.

DeMerit: the tanh function still suffers from the vanishing gradient problem. During backpropagation, the gradients of the tanh function can become very small (close to zero). This issue is particularly problematic for deep networks with many layers.



x	-5	-3	-1	0	1	3	5
f(x)	-0.999	-0.995	-0.761	0	0.761	0.995	0.999

Hard Tanh Function

The Hard Tanh activation function is a computationally efficient, piecewise linear approximation of the standard Hyperbolic Tangent (tanh) function. It is designed to offer similar properties to (tanh) (zero-centered output), range of [-1, 1] while reducing the computational cost associated with exponential calculations.

Mathematical Definition: The Hard Tanh activation function is defined as:

$$\text{HardTanh}(x) = \begin{cases} 1 & \text{if } x > 1 \\ -1 & \text{if } x < -1 \\ x & \text{otherwise} \end{cases}$$

Key Properties:

- **Computational Efficiency:** It is much faster to compute than the standard **tanh** function because it uses simple comparisons and clamping (min/max operations) rather than exponentiation.
- **Linear Regions:** It behaves linearly within the range **[-1, 1]** and saturates (becomes flat) outside of it.
- **Zero-Centered:** Like **tanh**, it is also zero-centered, which can help accelerate training.

Hard Tanh Function

Advantages and Disadvantages:

Advantages:

- **Speed:** Ideal for resource-constrained environments like embedded systems or IoT devices.
- **Stabilization:** Bounded outputs can prevent exploding activations.
- **Quantization Friendly:** The piecewise linear nature works well with low-bit, fixed-point integer representations.

Disadvantages:

- **Saturating Derivative:** The gradient is zero when **$x < -1$** or **$x > 1$**, which can lead to vanishing gradient problems in very deep networks.
- **Less Smooth:** As a piecewise linear function, it is not differentiable at **$x=-1$** and **$x=1$.**

Common Applications:

- **Hidden Layers:** Used in hidden layers of neural networks to maintain zero-centered data.
- **Edge AI:** Employed in low-latency and low-power inference scenarios.
- **Recurrent Neural Networks (RNNs):** Sometimes used in RNNs where linear approximations are preferred for speed.
- **Usage in Framework:** In PyTorch, can be used to apply this function, often with customizable min and max values. Similarly, it is available in other deep learning frameworks as a faster alternative.

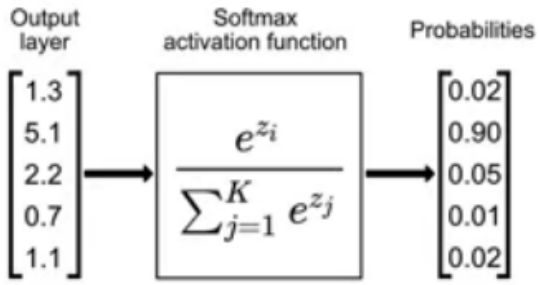
Softmax Function

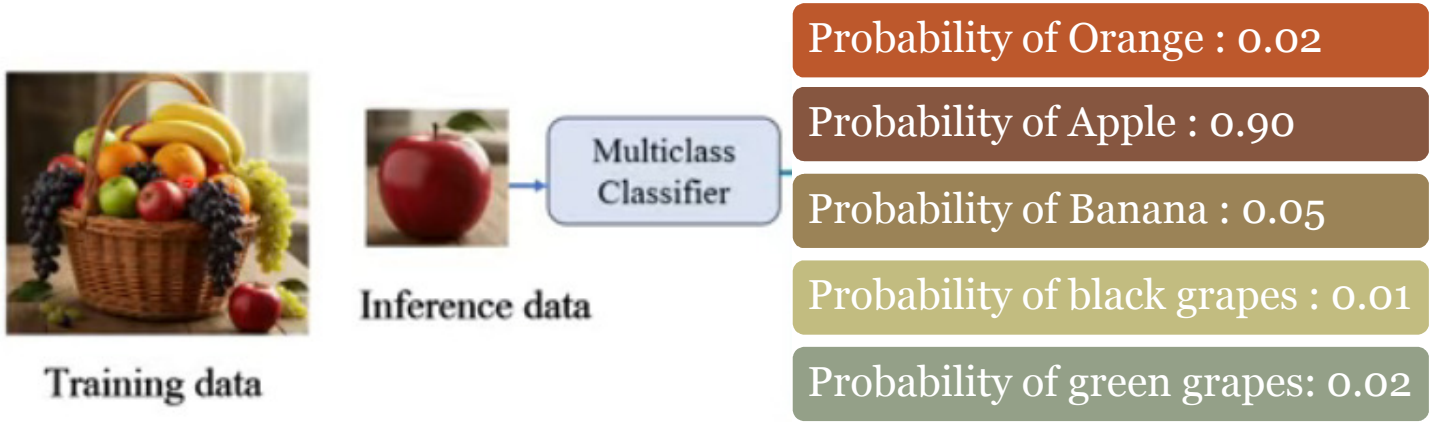
Softmax function is designed to handle multi-class classification problems.

- Softmax is a non-linear activation function.
- It transforms raw output scores from a neural network into probabilities.
- The Softmax function ensures that each class is assigned a probability, helping to identify which class the input belongs to.
- It works by squashing the output values of each class into the range of 0 to 1 while ensuring that the sum of all probabilities equals 1.
- It operates on a vector.

Types of Activation Function

Softmax function is defined as:





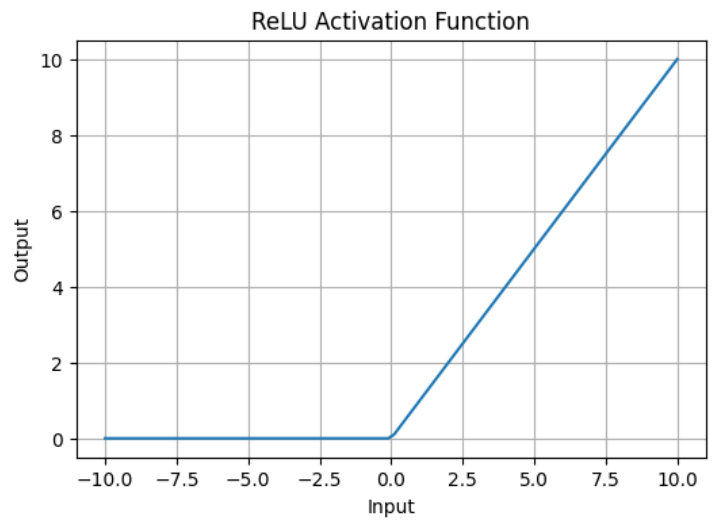
ReLU Activation Function

ReLU Activation function: The Rectified Linear Unit (ReLU) activation function has the form: f(x) = max (0,x).

Value Range: [0,∞),meaning the function only outputs non-negative values.

Nature: It is a non-linear activation function, allowing neural networks to learn complex patterns and making backpropagation more efficient.

Advantage over other Activation: ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.



Types of Activation Function

Drawback of ReLU Function:

Dying ReLU Problem:

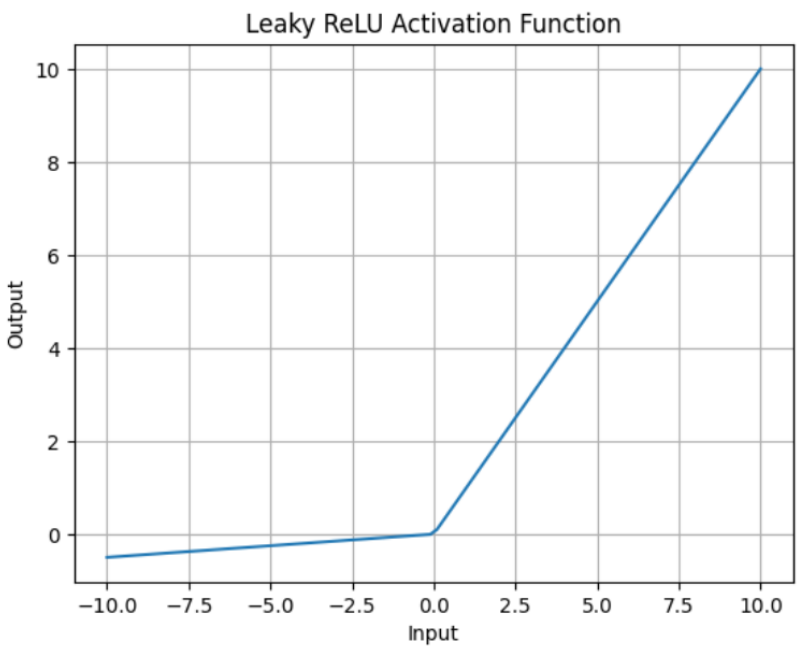
- Neurons can sometimes become inactive and only output Zero(0).
- This happens when large negative inputs results in zero gradient, leading to neurons that never activate and cannot learn further.

The above problem is solved by **Leaky ReLU Activation function** as shown in figure.

The activation function is defined as

f(x) = x , if x > 0

= 0.01 * x , if x <=0



Choosing the Right Activation Function

The choice of activation function depends on the type of problem you are trying to solve. Here are some guidelines:

For binary classification

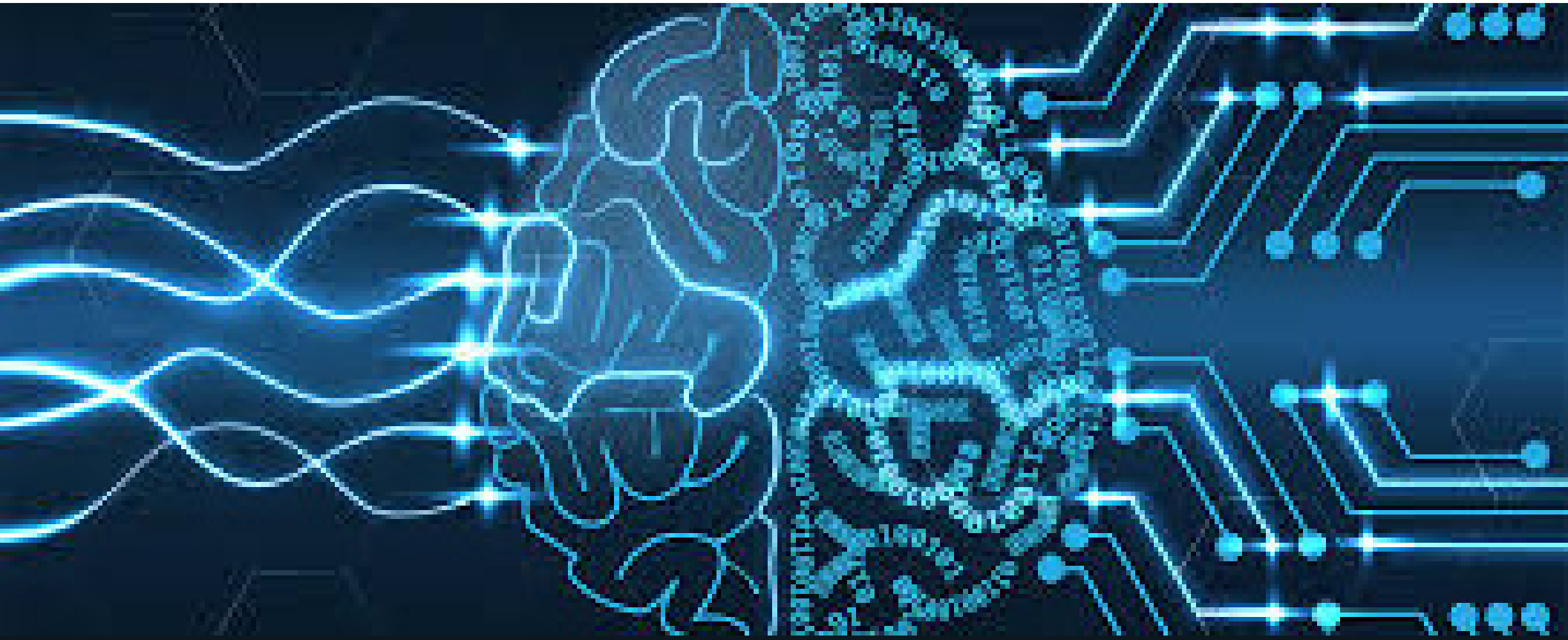
Use the sigmoid activation function in the output layer. It will squash outputs between 0 and 1, representing probabilities for the two classes.

For multi-class classification

Use the softmax activation function in the output layer. It will output probability distributions over all classes.

If unsure

Use the ReLU activation function in the hidden layers. ReLU is the most common default activation function and usually a good choice.



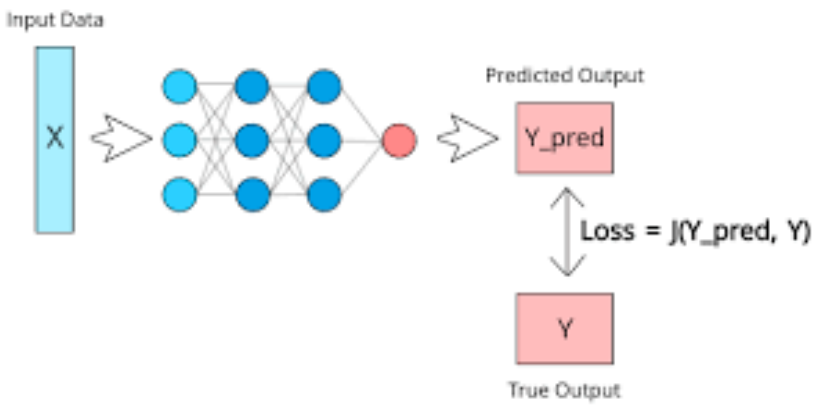
LOSS FUNCTION

DEEP LEARNING

Loss Function

A **loss function** in deep learning is a mathematical function that quantifies the error between a model's predicted output and the actual (true) target value.

It serves as a compass for the optimization process, providing a numerical measure that the model aims to minimize during training.

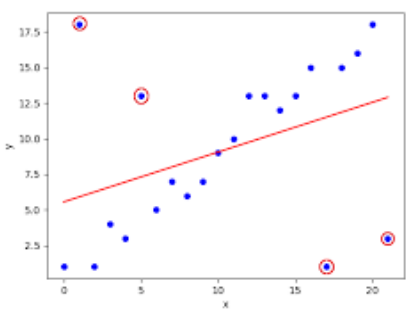


Loss Function

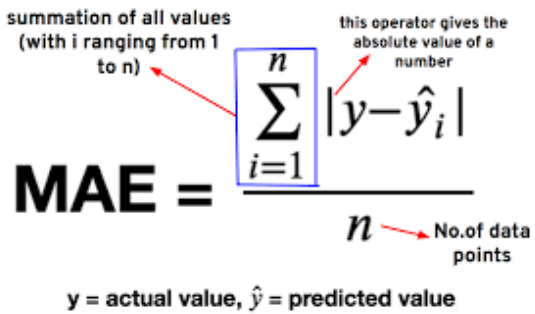
Mean Squared Error (MSE)

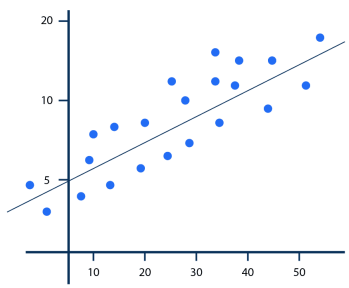
$$MSE = \frac{1}{N}\sum_{i}^{N}(Y_i - \hat{Y}_i)^2$$

MSE is sensitive to outlier



Mean Absolute Error (MAE)





Loss Function

Mean squared log error loss:

It's defined mathematically as :

$$MSLE = \frac{1}{n}\sum_{i=1}^{n}(log(1+\hat{y}_i) - log(1+y_i))^2$$

$$\frac{1}{N}\Sigma_{i=1}^{N}\Sigma_{j=1}^{M}\left(\log \widehat{y_{ij}} - \log y_{ij}\right)^2$$

Mean squared logarithmic error (MSLE) measures the squared difference between the logarithms of predicted and actual values, making it suitable for targets with different orders of magnitude and reducing the impact of large outliers.

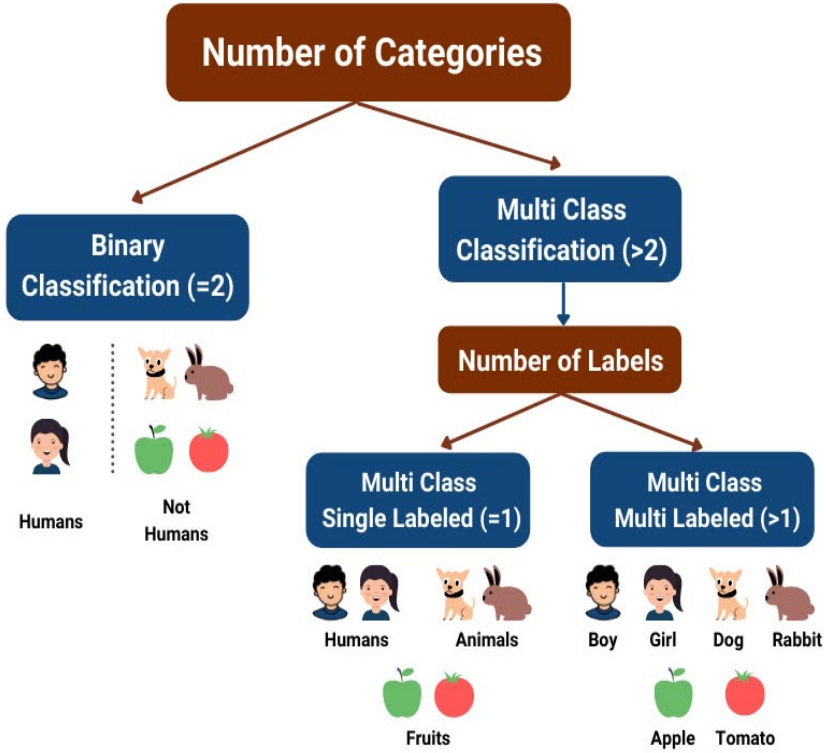
Mean absolute percentage error loss:

One of the most common metrics of model prediction accuracy, mean absolute percentage error (MAPE) is the percentage equivalent of mean absolute error (MAE). Mean absolute percentage error measures the average magnitude of error produced by a model.

$$\text{MAPE} = 100\frac{1}{n}\sum_{t=1}^{n}\left|\frac{A_t - F_t}{A_t}\right|$$

Where A_t is the actual value and F_t is the forecast value. Their difference is divided by the actual value A_t

Loss Function for Classification Problem



Binary Cross Entropy Loss

Binary Classification

Spam
Not Spam

Binary Cross Entropy Loss

Number of output nodes = 1

- Binary_Cross-Entropy Loss is also known as Log Loss.
- It is used for binary classification problems. It uses Sigmoid AF.
- It measures the performance of a classification model whose output is a probability value between 0 and 1.
- Mathematically Given as:

$$\text{Binary Cross-Entropy} = -\frac{1}{n} \quad {}_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

where:

- n is the number of data points
- y_i is the actual binary label (0 or 1)
- $\hat{y}_i$ is the predicted probability.